

UNIVERSIDAD ESPÍRITU SANTO

NOMBRE DE LA ASIGNATURA:

ESTRUCTURA DE DATOS

TEMA:

PROYECTO: LISTA DE CONTACTO

AUTORES:

KAREL LÁZARO GONZÁLEZ RUÍZ

JUSTIN AARÓN SOLEDISPA DUEÑAS

JUAN DIEGO SOTOMAYOR JIMÉNEZ

DOCENTE:

MARISOL VILLACRES

GUAYAQUIL-ECUADOR

2026

1. Organización Modular del Sistema

Se estructuró el proyecto separando claramente las responsabilidades para facilitar el mantenimiento y la prueba unitaria:

1. Contacto: Representa la entidad de negocio. Encapsula los datos personales y la metadata operativa (frecuencia).
2. APrefijo<E>: Implementación pura del Árbol de Prefijos. No conoce de interfaz de usuario, solo gestiona nodos y caracteres.
3. Agenda: Actúa como fachada. Coordina la inserción múltiple (por nombre, apellido, apodo) y filtra resultados duplicados antes de mostrarlos.
4. Utilidades (Heap, ComparadorFrecuencia): Componentes reutilizables para algoritmos de ordenamiento sin depender de librerías externas de colección.

2. Implementación del TDA APrefijo

El TDA APrefijo constituye el motor de búsqueda central del proyecto. A diferencia de un árbol de búsqueda tradicional, no almacena palabras enteras en sus nodos, sino que desglosa cada cadena de texto carácter por carácter, creando un "camino". Esto permite búsquedas por coincidencia parcial extremadamente rápidas. A continuación, se detallan sus métodos y su comportamiento interno:

- Constructor (APrefijo()): Inicializa el nodo instanciando un mapa (HashMap) para almacenar las referencias a los nodos hijos (lo que permite un acceso constante al siguiente carácter) y una lista (LinkedList) vacía. La bandera booleana finNombre arranca en false.
- insertar(String palabra, E dato): Es el encargado de construir el árbol. Recibe una palabra y el objeto que debe almacenar. Recorre la palabra carácter por carácter; si el carácter no existe como clave en el mapa de hijos del nodo actual, crea un nuevo nodo. Al llegar al último carácter, marca ese nodo final con finNombre = true y guarda el objeto dentro de su lista. En el contexto del proyecto, se usa para guardar una referencia del contacto bajo su nombre, su apellido y su apodo.
- buscarPorPrefijo(String prefijo): Método público para iniciar la búsqueda. Coordina otros dos métodos: primero llama a buscarNodoFinal para ubicarse exactamente donde termina el prefijo escrito por el usuario, y luego delega la extracción a recolectarDatos.
- buscarNodoFinal(String prefijo): Método auxiliar de navegación pura. Desciende rama por rama siguiendo los caracteres del prefijo. Si en algún punto el carácter solicitado no existe, detiene la búsqueda y retorna null, lo que significa que ninguna palabra guardada coincide.
- recolectarDatos(APrefijo<E> nodo, LinkedList<E> lista): Es un método recursivo fundamental para lograr el "autocompletado". A partir del nodo final de un prefijo,

este método viaja por absolutamente todas las ramificaciones subyacentes. Cada vez que detecta un nodo marcado con `finNombre`, añade sus datos a la lista acumulativa.

- `eliminarContacto(String palabra, E dato)`: Coordina la eliminación. Primero localiza el nodo final de la palabra, luego remueve el dato de la lista interna llamando a `removeDato`. Si la eliminación fue exitosa, dispara el método `podarRama` para limpiar la estructura del árbol.
- `removeDato(APrefijo<E> nodo, E dato)`: Solo se enfoca en sacar el objeto específico de la `LinkedList` del nodo. Como un mismo nombre puede pertenecer a varios contactos, solo borra el objeto exacto. Si la lista queda completamente vacía tras borrarlo, desmarca el flag `finNombre`.
- `podarRama(APrefijo<E> nodo, String palabra, int indice)`: Representa una optimización crítica de memoria. Utiliza recursividad de retorno para "subir" por el árbol (desenrollando la pila de llamadas) verificando si los nodos ancestros han quedado sin descendientes y sin datos. Si es así, elimina las claves del `HashMap`, evitando dejar nodos "fantasma" que solo ocuparían RAM innecesariamente.

3. Uso Estratégico de Comparadores y Heap

- El TDA Heap se implementó para gestionar el ordenamiento eficiente y dinámico de los contactos, ignorando por completo qué objetos está manejando gracias al uso de Genéricos y la interfaz `Comparator`.
- Constructor (`Heap(Comparator<E> cmp, boolean ordenAsc)`): Inicializa el arreglo dinámico (`ArrayList`). Un detalle técnico importante es que inserta un elemento null en el índice 0; esto se hace para alinear los índices con las fórmulas matemáticas tradicionales de los árboles binarios.
- Cálculo de Índices (`idxIzq`, `idxDer`, `idxPadre`): Métodos matemáticos base que localizan la posición relativa en el arreglo basándose en multiplicaciones (hijo izquierdo: $2*r$, hijo derecho: $2*r+1$, padre: $r/2$). Si el resultado supera el tamaño del Heap, retornan -1.
- `estaVacio()` y `esHoja(int indice)`: Evaluaciones de estado. `esHoja` es vital para detener los procesos recursivos, ya que comprueba si un nodo carece de descendencia (índices hijos en -1).
- `tienePrioridad(E candidato, E actual)`: Es el puente entre el TDA y la lógica del negocio. Delega la decisión de quién es "mayor o menor" al `Comparator` que se le inyectó, y respeta la preferencia de orden ascendente o descendente.
- `intercambio(int i, int j)`: Un método auxiliar clásico que realiza un intercambio de valores entre dos posiciones del arreglo usando una variable temporal.
- `insertar(E elemento)` y `flotar(int indice)`: Todo elemento nuevo ingresa en la última posición disponible del arreglo. Posteriormente, el método `flotar` evalúa recursivamente si ese nuevo elemento tiene mayor prioridad que su "padre". De ser así, intercambia sus posiciones repetidas veces hasta que el elemento encuentre su lugar jerárquico correspondiente.

- `extraer()` y `ajustar(int indice)`: El método `extraer` siempre retira y retorna la raíz del árbol (el elemento con máxima prioridad). Para rellenar el vacío dejado en la cima, traslada el último elemento de la estructura a la raíz y llama al método `ajustar`. Este último hunde el elemento recursivamente comparándolo con sus hijos, restableciendo la propiedad del montículo (Heapify).
- `ordenarConHeap(LinkedList<E> lista, Comparator<E> cmp, boolean asc)`: Es un método estático y utilitario que actúa como un clasificador integral. Recibe una lista desordenada, inserta cada uno de sus elementos en un Heap temporal, y luego extrae repetitivamente la raíz para poblar y retornar una nueva lista perfectamente ordenada.

4. Arquitectura y Métodos de las Clases de Negocio

4.1 Clase Contacto

- Representa la entidad fundamental de datos del proyecto, encapsulando la información del usuario de manera segura.
- Constructor y Getters: Almacenan los atributos asegurándose de limpiarlos de espacios en blanco excedentes usando `.trim()`. Los getters (`getNombre`, `getTelefonoMovil`, etc.) exponen esta información a la interfaz visual y a los comparadores de ordenamiento.
- `incrementarFrecuencia()`: Aumenta el contador interno frecuencia en 1. Este método es el pilar que permite al sistema "aprender" de los hábitos del usuario, priorizando contactos buscados asiduamente.
- `equals(Object obj)` y `hashCode()`: Son sobrescritos de la clase `Object` para definir la identidad única del contacto. Consideran que dos contactos son la misma persona únicamente si su nombre, apellido, apodo y teléfono móvil son exactamente iguales. Es vital para que la clase `Agenda` evite duplicados visuales.
- `toString()`: Proporciona el formato de texto estandarizado para imprimir el contacto de forma legible por pantalla.

4.2 Clase Agenda

- Actúa como el coordinador principal. Su propósito es ocultar la complejidad del Árbol de Prefijos al resto del programa.
- Constructor (`Agenda()`): Inicializa el `APrefijo` en memoria y crea un `HashSet` llamado `archivosCargados`.
- `archivoYaFueCargado(Path ruta)` y `marcarArchivoComoCargado(Path ruta)`: Operan como un registro histórico. Convierten la ruta de los archivos importados a minúsculas absolutas y validan que un archivo CSV no se ingrese dos veces para evitar la duplicación accidental y masiva de la base de datos.
- `insertarAtributo(String valor, Contacto c)` y `eliminarAtributo(...)`: Métodos privados de validación que se aseguran de que no se inserten cadenas nulas o vacías en el árbol, y transforman los textos a minúsculas para mantener consistencia en la búsqueda.

- registrarContacto(Contacto c): Multiplica la indexación. Por cada contacto nuevo, invoca la inserción en el árbol tres veces distintas: con su nombre, su apellido y su apodo.
- eliminarContacto(Contacto c): Realiza la operación inversa al registro. Elimina de manera segura las tres ramas de acceso (nombre, apellido y apodo) asociadas a dicho objeto dentro del Árbol Trie.
- obtenerUnicos(String prefijo): Soluciona un problema lógico de la indexación múltiple. Si el usuario busca "Ma" y el contacto es "María Macías", el árbol devolverá el objeto por el nombre y también por el apellido. Este método introduce los resultados en un HashSet para limpiar y purgar los duplicados, entregando una lista única.
- buscarParaEliminar(String prefijo): Obtiene los contactos únicos asociados al prefijo y los devuelve ordenados por frecuencia usando el Heap, permitiendo al usuario seleccionar fácilmente a quién borrar.
- buscarPorPrefijoConFrecuencia(String prefijo): Es el método núcleo de búsqueda interactiva. Encuentra los contactos, recorre la lista aplicando el método incrementarFrecuencia() a cada uno, y finalmente retorna los resultados organizados por prioridad usando el Montículo.
- obtenerTodosLosContactos(): Aprovecha la lógica del Árbol Trie mandando a buscar un prefijo vacío (""). Esto obliga a recolectar todos los datos existentes en la estructura. Luego, devuelve el catálogo completo ordenado usando Heap en conjunto con el comparador alfabético.

4.3 Clases de Ordenamiento

Dado que el TDA Heap fue diseñado de manera genérica, requiere instrucciones externas para saber cómo comparar dos objetos Contacto. Estas clases implementan la interfaz Comparator<Contacto> y establecen reglas estrictas, incluyendo múltiples niveles de desempate.

- ComparadorAlfabetico: Su utilidad principal es organizar la libreta completa de la A a la Z. Compara primero por nombre usando compareToIgnoreCase para ignorar mayúsculas. Si los nombres son idénticos (empate), evalúa el apellido, y si el empate persiste, evalúa el apodo.
- ComparadorFrecuencia: Es el cerebro detrás de las sugerencias inteligentes. Prioriza el atributo frecuencia del contacto (quién ha sido buscado más veces). Si dos contactos tienen la misma frecuencia de búsqueda, aplica hasta tres criterios de desempate en orden alfabético: Nombre, Apellido, Apodo y, en última instancia, el Número de Teléfono Móvil, garantizando que el Heap nunca tenga ambigüedades al ordenar.

4.4 Clase GestorAgenda

- Esta clase actúa como un escudo o capa de seguridad para la entrada de datos (Input/Output). Su utilidad es fundamental para evitar que el programa colapse por

errores de tipeo del usuario y para asegurar la integridad de la información que ingresa al Árbol Trie.

- Validación mediante Expresiones Regulares (Regex): Cuenta con métodos como `esTelefonoValido` y `esCorreoValido`. El primero elimina espacios y guiones, verificando matemáticamente que la cadena resultante tenga entre 7 y 15 dígitos. El segundo asegura que el correo contenga el formato universal (texto + @ + dominio + extensión).
- Ciclos de Validación (`leerTelefono`, `leerCorreo`, `leerCriterioBusqueda`): Estos métodos implementan bucles `while(true)`. Si el usuario ingresa caracteres especiales no permitidos o formatos inválidos, el gestor atrapa el error, muestra un mensaje de advertencia y vuelve a solicitar el dato sin que el programa principal se detenga (evitando excepciones en tiempo de ejecución). Además, maneja lógicas de cancelación (escribir "cancelar") o saltos (dar Enter para campos vacíos).

4.5 Clase MenuAgenda

- Representa la capa de presentación (Vista) en el entorno de consola. Su función es separar completamente la interacción con el usuario de la lógica de procesamiento y almacenamiento.
- `mostrarMenuPrincipal()`: Despliega la interfaz de texto e invoca a las distintas opciones mediante un bloque switch. Dependiendo de la opción elegida, delega la acción a métodos específicos como `menuBusqueda`, `menuRegistro`, `menuEliminacion`, etc.
- No realiza cálculos lógicos ni manipulaciones de estructuras directamente. Su utilidad es solicitar información al usuario a través del `GestorAgenda`, enviar esos datos a la clase `Agenda` para que los procese en el Árbol Trie o el Heap, y finalmente recibir los resultados (como una lista enlazada) para formatearlos visualmente en pantalla mediante el método `imprimirLista()`.

4.6 Clase CargadorDatos

- Esta clase es responsable de la persistencia de datos (E/S de archivos). Separa la complejidad del manejo de archivos del resto del programa, permitiendo la interoperabilidad mediante archivos estandarizados CSV (Valores Separados por Comas).
- `cargar()` y `cargarSilencioso()`: Administran la importación de contactos. Usan la librería `java.nio.file.Path` para resolver las rutas absolutas y relativas. Leen el archivo CSV línea por línea usando `BufferedReader`, dividen la información separada por comas y construyen nuevos objetos `Contacto` que luego inyectan masivamente en la `Agenda`. El método "silencioso" es útil para cargar archivos predeterminados al iniciar el sistema sin saturar la consola con mensajes.
- `exportarAgenda()`: Realiza el proceso inverso. Recorre el Árbol de Prefijos recolectando todos los contactos, los ordena (aprovechando las utilidades de ordenamiento del sistema) y escribe la información en disco utilizando

BufferedWriter, asegurando de manejar y advertir sobre posibles excepciones IOException si la ruta de destino no existe o no tiene permisos de escritura.

4.7 Clase PantallaAgenda

- Esta es la clase principal o "punto de entrada" (Entry Point) del programa, ya que contiene el método ejecutable main. Su utilidad es inicializar los cimientos del sistema y definir el entorno de ejecución.
- main(String[] args): Al arrancar, este método instancia por primera vez la Agenda (que a su vez crea el Árbol Trie) y el GestorAgenda. Luego, presenta un submenú inicial muy importante que define el flujo del proyecto, permitiendo al usuario elegir entre dos modos:
- Modo Consola: Si el usuario elige la opción 1, se instancia la clase MenuAgenda, inyectándole la agenda creada, y se inicia el bucle de texto tradicional.
- Interfaz Gráfica: Si el usuario elige la opción 2, la clase actúa como puente hacia JavaFX. Asigna la agenda instanciada a una variable estática compartida (AppJavaFX.agendaCompartida) y ejecuta Application.launch() para levantar las ventanas gráficas. Esto demuestra una excelente práctica de diseño, probando que el "Motor" (Agenda/Árbol) funciona perfectamente sin importar si se usa texto plano o botones visuales.

4.8 Clase ServicioExportacion

- Actúa como una clase utilitaria dedicada exclusivamente a gestionar la logística de salida de archivos. Separa las validaciones de directorios y rutas del proceso de escritura real, manteniendo el código limpio.
- exportarEnCarpetaPredeterminada(Agenda agenda): Es una función de exportación rápida. Si el usuario no quiere lidiar con rutas de carpetas, este método guarda automáticamente un archivo llamado "contactos_exportados.csv" en la raíz del proyecto, delegando la escritura a CargadorDatos.
- prepararRuta(String ruta): Es el método de validación y formateo de texto más riguroso para la exportación. Si el usuario envía una ruta vacía, asigna el nombre por defecto. Si el usuario ingresa solo la ruta de una carpeta, le añade automáticamente el nombre del archivo al final. Además, verifica matemáticamente que la cadena termine en .csv (agregándolo si falta) y comprueba a nivel de sistema operativo si el directorio "padre" donde se quiere guardar el archivo realmente existe.
- archivoExiste(String ruta): Un método de seguridad (booleano) que utiliza la clase File de Java para revisar el disco duro. Su utilidad es alertar al MenuAgenda si ya hay un archivo con ese nombre para que el programa le pregunte al usuario si desea sobrescribirlo, evitando pérdidas accidentales de datos.
- exportar(Agenda agenda, String ruta): Es un método puente. Una vez que la ruta ha sido validada y confirmada, toma la agenda y la ruta limpia, y llama al motor de escritura real (CargadorDatos.exportar).

4.9 Clases de Interfaz Gráfica (JavaFX)

Este conjunto de clases representa la capa de presentación visual del proyecto. Siguen estrictamente el principio de separación de responsabilidades, asegurando que el diseño de las pantallas no interfiera con la lógica de negocio (el Árbol Trie y el Heap), sino que simplemente se comuniquen con ella.

- **AppJavaFX:** Es el motor de arranque del entorno gráfico. Hereda de la clase `Application` de JavaFX y configura la ventana principal (`Stage`). Una de sus utilidades más importantes es implementar un patrón de acceso global mediante el método `getAgendaCompartida()`, garantizando que todas las pantallas de la interfaz interactúen siempre con la misma instancia que en memoria.
- **VistaBienvenida:** Actúa como el contenedor principal y el enrutador del sistema. Está construida sobre un `BorderPane`, donde el lado izquierdo alberga un menú de navegación (`sidebar`) interactivo, y el centro es un espacio dinámico. Su utilidad es intercambiar las vistas operativas en el centro de la pantalla según lo que presione el usuario, dando la sensación de una aplicación fluida de una sola ventana (`Single Page Application`).
- **VistaRegistro:** Es el formulario interactivo para crear contactos. Su valor añadido radica en que realiza validaciones visuales previas (usando las reglas de `CargadorDatos` para validar teléfonos y correos) y notifica al usuario con mensajes de error o éxito coloreados antes de permitir que la información llegue al núcleo de la agenda.
- **VistaBúsqueda y VistaMostrar:** Son las encargadas de la renderización de datos. Toman las listas enlazadas (devueltas por los algoritmos de búsqueda y ordenamiento) y las transforman dinámicamente en elementos gráficos atractivos (`Tarjetas`). Utilizan combinaciones de contenedores `HBox` y `VBox` junto con estilos CSS integrados para añadir sombras, tipografías legibles y colores a cada resultado.
- **VistaEliminación:** Combina la visualización de contactos con acciones críticas. Genera tarjetas interactivas que incluyen un botón de borrado directo. Para proteger la integridad de los datos, invoca diálogos emergentes de seguridad (`Alert` de tipo confirmación) para asegurar que el usuario realmente desea eliminar el registro antes de ejecutar la acción.
- **VistaCarga y VistaExportación:** Son los puentes gráficos hacia el disco duro del usuario. En lugar de obligar al usuario a escribir rutas de texto (como en el modo consola), utilizan la herramienta `FileChooser` de JavaFX. Esto abre el explorador de archivos nativo del sistema operativo (`Windows/Mac`) para buscar o guardar archivos `.csv` de forma visual, manejando alertas automáticas si se intenta sobrescribir un archivo existente.

5. Presentación dual: Consola e Interfaz Gráfica

El proyecto incluye una la incorporación de una arquitectura dual:

- Modo consola

- Modo gráfico (JavaFX).

Ambos entornos reutilizan exactamente las mismas estructuras y lógica de negocio. La interfaz gráfica no contiene reglas adicionales, demostrando que el núcleo del sistema fue diseñado de manera independiente de la capa de presentación.

6. Generación de documentos

El sistema permite:

- Exportación en formato CSV con nombre y ruta preferida en consola.
- Exportación en formato CSV con nombre y ruta preferida en Interfaz Gráfica.

7. Fuentes consultadas y uso de IA

El desarrollo del proyecto se basó principalmente en el material proporcionado en clase, especialmente en los temas relacionados con:

- Utilidad del Heap
- Recorridos recursivos con metodos privados.
- Comparadores de caracteres.

La inteligencia artificial se utilizó como herramienta de apoyo para:

- Resolver dudas específicas de implementación.
- Construcción de interfaces con JavaFX.
- Validar decisiones de diseño.
- Mejorar la organización del código.
- Revisar coherencia de los datos.

Su uso fue complementario y enfocado en aclarar conceptos o confirmar buenas prácticas, no en reemplazar el desarrollo propio del proyecto.